

PetPro Connect — Project Brief for Claude Code

Place this file as `CLAUDE.md` in the root of your repo. Claude Code reads it automatically for project context at the start of sessions.

What This Project Is

PetPro Connect is a business management platform for professional dog walkers (Phase 1), architected to expand to trainers, groomers, sitters, and boarding facilities later without redesign. Solo founder project, currently in Phase 1 build (8-week timeline).

Hard Constraints — Do Not Violate

- **Modular monolith.** No microservices. Single Node.js/Express deployable.
- **REST only.** No GraphQL.
- **Stripe only** for payments.
- **eSign must go through the `IeSignProvider` adapter interface** — never call the Nitro Sign API directly from service code. `ContractService` should only ever talk to `IeSignProvider`.
- **Events table is append-only.** Never write UPDATE or DELETE against `events` — only INSERT. A database trigger already enforces this; don't try to work around it.
- **Signed contracts are immutable.** Once `contracts.status = 'signed'`, `generated_html` must never change.
- **Follow the existing type definitions in `src/types/index.ts`.** If a field doesn't exist there, either it needs to be added deliberately (flag it to me) or the approach is wrong.

Marketplace Seams — Preserve These

The core product is built so a future Phase 3 marketplace can plug in without touching core modules. When adding features, keep these intact:

1. **Typed accounts** — `account_type` enum drives auth/permissions, never hardcode role checks
2. **Append-only event log** — significant actions get published as events
3. **Provider adapters** — eSign and (later) maps go through interfaces, never a vendor SDK directly in service code
4. **Generic billing** — Stripe products are "billable items attached to an account," not hardcoded to walker subscriptions

Current Stack

- **Runtime:** Node.js 20+, TypeScript (strict mode)
- **Database:** Supabase (Postgres) — use Supabase client where reasonable, raw `pg` queries are also fine for complex queries
- **Auth:** Supabase Auth (magic-link for owners, email/password for professionals) — **not** custom JWT, this replaces what may appear in early scaffolding
- **Hosting:** Render
- **Storage:** Supabase Storage (signed PDFs, images) — not AWS S3
- **Payments:** Stripe (test mode during build)
- **eSign:** Nitro Sign (via `IeSignProvider` adapter)
- **Email:** Resend or SendGrid (free tier)

Build Order (8-week plan — see `8_WEEK_ROADMAP.md`)

1. Auth + deployment foundation
2. CRM (clients, pets)
3. Contracts — generation + in-person signing
4. Contracts — Nitro Sign electronic signing
5. Payments — Stripe integration
6. Scheduling — services + appointments
7. Messaging + email notifications
8. Owner portal + end-to-end testing

Currently working on: *(update this line each week so Claude Code knows where you are)*

Working Preferences

- I'm a beginner programmer. Explain non-obvious decisions briefly as you make them — I don't need a tutorial, but I do need to understand what was built well enough to make product calls.
- Prefer one feature fully working (with a quick manual test I can run) over multiple features half-done.
- Flag it clearly if a request would violate one of the hard constraints above, rather than quietly working around it.
- Commit to git after each working feature with a clear message.

Reference Files in This Repo

- `src/types/index.ts` — source of truth for all data shapes
- `src/db/migrations/` — schema, run in order 001-010
- `src/integrations/esign/` — provider adapter pattern (Nitro Sign implemented)
- `src/services/AccountService.ts`, `ContractService.ts`, `EventService.ts` — reference pattern for how services should be structured (repeat this pattern for ClientService, SchedulingService, PaymentService, MessagingService, NotificationService)
- `ARCHITECTURE.md` — full system design and data flow
- `8_WEEK_ROADMAP.md` — week-by-week scope